

09 — Free Space Map (FSM)

Module: PostgreSQL Internals | **Difficulty:** Advanced | **Est. reading time:** 35 min

Table of Contents

1. [Learning Objectives](#)
 2. [Overview](#)
 3. [FSM Structure: The AVL-Like Tree](#)
 4. [How INSERT and HOT UPDATE Use the FSM](#)
 5. [FILLFACTOR: Reserving Space for Updates](#)
 6. [HOT Updates: Heap Only Tuple](#)
 7. [How VACUUM Updates the FSM](#)
 8. [FSM File Structure](#)
 9. [ASCII Diagrams](#)
 10. [Live Observation Queries](#)
 11. [Common Mistakes](#)
 12. [Best Practices](#)
 13. [Performance Considerations](#)
 14. [Interview Questions](#)
 15. [Exercises with Solutions](#)
 16. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Describe the tree structure of the FSM and how it provides $O(\log n)$ page selection.
 - Explain what FILLFACTOR does and calculate the ideal FILLFACTOR for a given workload.
 - Explain the HOT (Heap Only Tuple) update mechanism and the conditions required for it.
 - Describe how VACUUM and INSERT interact with the FSM.
 - Use `pg_freespace` and `pg_relation_size(t, 'fsm')` to inspect FSM state.
 - Determine when a stale FSM leads to unnecessary table extension.
-

Overview

The **Free Space Map (FSM)** is a compact per-relation structure that tracks how much free space is available in each heap page. It is stored in the `_fsm` fork of a relation file.

Without the FSM:

- `INSERT` would have to read page after page looking for one with enough free space.
- The only reliable option would be extending the file (adding new pages), causing unnecessary bloat.

With the FSM:

- `INSERT` consults the FSM to find a page with at least `tuple_size` bytes free.
 - The FSM lookup is $O(\log n)$ (tree traversal).
 - Pages with freed space from VACUUM are immediately reusable.
-

FSM Structure: The AVL-Like Tree

The FSM uses a **3-level binary tree** structure where each internal node holds the **maximum free space** available among its children. The leaves represent individual heap pages.

Each leaf stores one byte representing the available free space on that page:

```
Free space byte = floor(actual_free_bytes / 32)
```

So the byte value 0–255 maps to 0–8160 bytes of free space (in units of 32 bytes). The maximum representable free space is $255 \times 32 = 8160$ bytes (the usable portion of a full empty page, which is 8192 – 24 header bytes – some overhead).

Tree levels

```
Level 2 (root):  max free space across all pages in the relation
Level 1:         max free space across a subtree of pages
Level 0 (leaf):  free space in a single heap page (1 byte)
```

Each 8 KB FSM page stores a subtree covering exactly 4096 leaf pages (heap pages). For large tables, multiple FSM pages are chained.

Lookup algorithm

To find a page with at least `N` bytes free:

1. Start at the root.
2. Descend to the child with the higher free-space value if it meets the requirement.
3. Continue until reaching a leaf (a specific heap page number).
4. Return that page number to the caller.

This is $O(\log n)$ — much faster than a linear scan for free pages.

How INSERT and HOT_UPDATE Use the FSM

INSERT

```
INSERT INTO orders (customer_id, total) VALUES (42, 99.99);
```

1. Calculate the expected tuple size.
2. Ask the FSM for a page with at least `tuple_size` bytes free.
3. If the FSM returns a page: pin the page in `shared_buffers`, verify free space (FSM is approximate), insert the tuple.
4. If the FSM has no suitable page (all entries are 0 or insufficient): extend the relation by allocating a new page.
5. Update the FSM to reflect the reduced free space on the chosen page.

HOT_UPDATE

A HOT (Heap Only Tuple) update inserts the new tuple version on the **same page** as the old version:

1. Check if the page containing the old tuple has enough free space for the new version.

2. If yes: write new tuple on same page, set `HEAP_HOT_UPDATED` on old tuple, set `HEAP_ONLY_TUPLE` on new tuple.
3. No new index entries are needed (the index still points to the old tuple's slot, which redirects to the new version).

HOT updates require the page to have free space. `FILLFACTOR` ensures this.

FILLFACTOR: Reserving Space for Updates

FILLFACTOR is a storage parameter (0–100, default 100) that controls how full PostgreSQL tries to keep heap pages during INSERT operations:

```
-- Set fillfactor at table creation
CREATE TABLE orders (
    id      bigint GENERATED ALWAYS AS IDENTITY,
    status  text,
    total   numeric
) WITH (fillfactor = 70);

-- Or alter an existing table
ALTER TABLE orders SET (fillfactor = 70);
-- Takes effect for new pages only; VACUUM FULL or CLUSTER to apply to existing
pages
```

With `fillfactor = 70` :

- INSERTs fill pages to only 70% capacity (5734 bytes of 8192).
- The remaining 30% (2458 bytes) is reserved for future updates (HOT updates).
- The FSM marks pages above 70% as "full" for INSERT purposes.

Choosing FILLFACTOR

Workload	Recommended FILLFACTOR
INSERT-only / append-only	100 (default) — no space reserved
Low UPDATE rate	90
Medium UPDATE rate	70–80
High UPDATE rate (OLTP)	50–70
Data warehouse (few updates)	100

Lower FILLFACTOR:

- More HOT updates → less index bloat, less WAL for index updates.
 - Less data per page → more pages, more I/O for sequential scans.
-

HOT Updates: Heap Only Tuple

HOT (Heap Only Tuple) is a critical optimisation that avoids writing to indexes on UPDATE when:

1. The updated row version fits on the **same page** as the old version.
2. None of the **indexed columns** were modified.

HOT update chain

```
Old tuple (offset 1): t_xmax=200, HEAP_HOT_UPDATED, t_ctid=(0,3)
                        |
                        | HOT chain
                        ▼
New tuple (offset 3): t_xmin=200, HEAP_ONLY_TUPLE, t_ctid=(0,3) [self]
```

The index still points to ItemId[1]. When an index scan finds slot 1:

- If slot 1 is HEAP_HOT_UPDATED: follow the HOT chain to slot 3 (the live version).
- This avoids writing a new index entry entirely.

Benefits of HOT

- **No index bloat:** Index entries are not multiplied on every update.
- **Less WAL:** Index WAL records are not written for HOT updates.
- **Less work for VACUUM:** Index cleanup is simpler (VACUUM can prune HOT chains during heap cleanup, without a full index scan).

HOT chain cleanup (HOT pruning)

Even without running a full VACUUM, PostgreSQL "prunes" HOT chains during page access:

- If the old tuple in a HOT chain is dead to all running transactions, it is cleaned up immediately when the page is read.
- The ItemId becomes `LP_REDIRECT` pointing directly to the live version.
- This keeps page space clean without waiting for VACUUM.

How VACUUM Updates the FSM

After VACUUM removes dead tuples from a page, it updates the FSM:

```
Before VACUUM:
Page 7: pd_upper=6800, pd_lower=48
Free space = 6800 - 48 = 6752 bytes
FSM byte = floor(6752 / 32) = 211
```

```
After VACUUM (dead tuples removed, more free space):
Page 7: pd_upper=7400, pd_lower=48
Free space = 7400 - 48 = 7352 bytes
FSM byte = floor(7352 / 32) = 229
```

```
VACUUM updates FSM leaf for page 7 to 229
Then propagates the change up the tree
```

After VACUUM updates FSM entries, new INSERT operations immediately benefit from the reclaimed space.

If VACUUM is never run (autovacuum disabled or lagging), the FSM shows pages as full even though they have dead-tuple space. INSERT extends the file unnecessarily.

FSM File Structure

Relation: base/16384/16385 (heap)
FSM fork: base/16384/16385_fsm (free space map)

FSM page structure (8 KB each):

FSM page header (standard PageHeaderData)
Internal tree nodes (1 byte each): Node 0: root (max of all) Nodes 1-2: level-1 nodes Nodes 3-6: level-2 nodes ...
Leaf nodes (1 byte each, 4096 leaves per FSM page): Leaf 0: free space in heap page 0 Leaf 1: free space in heap page 1 ... Leaf 4095: free space in heap page 4095

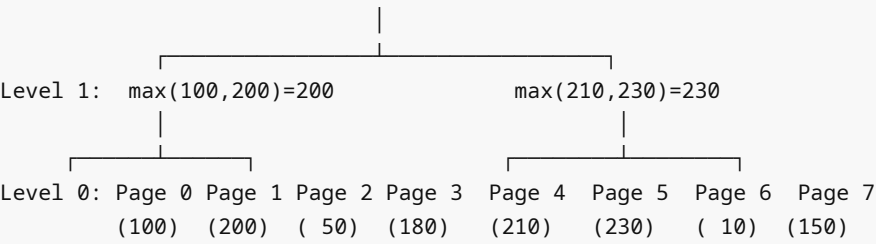
For a table with more than 4096 heap pages, the FSM has multiple 8 KB pages. There is one more level of tree structure to manage multiple FSM pages.

ASCII Diagrams

FSM Tree Structure

Heap pages 0-7 covered by this FSM subtree:

Level 2 (root): $\max(\text{Level1_left}, \text{Level1_right}) = \max(200, 230) = 230$



INSERT needing 180 bytes free:

$180 / 32 = 5.6 \rightarrow$ need FSM byte ≥ 6 ($6 \times 32 = 192$ bytes)

Start at root ($230 \geq 6$) $\rightarrow$ go right ($230 \geq 6$)

Level 1 right: $230 \geq 6 \rightarrow$ go to Level 0 right-right: Page 5 (230) ✓

$\rightarrow$ INSERT goes to Page 5

HOT Update vs Regular Update

Table: orders, index on (status)

Scenario 1 – Regular UPDATE (status column changed):

Old tuple: t_xmin=100, t_xmax=200, t_ctid=(5,3) → points to new tuple

New tuple: t_xmin=200, t_xmax=0, t_ctid=(5,4)

Index: entry for old status → remove

entry for new status → ADD ← index write required

Scenario 2 – HOT UPDATE (non-indexed column changed, same page):

Old tuple: t_xmin=100, t_xmax=200, HEAP_HOT_UPDATED, t_ctid=(5,3) → new tuple

New tuple: t_xmin=200, t_xmax=0, HEAP_ONLY_TUPLE, t_ctid=(5,3) [self]

Index: NO changes (index still has old entry → follows HOT chain to live tuple)

FSM: page 5 free space decreases

Live Observation Queries

```
-- 1. Install pg_freespace (bundled)
CREATE EXTENSION IF NOT EXISTS pg_freespacemap;

-- 2. View free space per page for a table
SELECT blkno, avail
FROM pg_freespace('orders')
ORDER BY avail DESC
LIMIT 10;

-- 3. Average free space
SELECT
    count(*)                AS total_pages,
    round(avg(avail), 0)     AS avg_free_bytes,
    pg_size_pretty(sum(avail)::bigint) AS total_free_space,
    round(100.0 * avg(avail) / 8192, 1) AS avg_free_pct
FROM pg_freespace('orders');

-- 4. FSM file size
SELECT pg_size_pretty(pg_relation_size('orders', 'fsm')) AS fsm_size;

-- 5. Check HOT update effectiveness
-- High ratio of hot_updates to n_tup_upd = HOT working well
SELECT
    relname,
    n_tup_upd          AS updates,
    n_tup_hot_upd      AS hot_updates,
    round(100.0 * n_tup_hot_upd / NULLIF(n_tup_upd, 0), 1) AS hot_pct
FROM pg_stat_user_tables
WHERE n_tup_upd > 0
ORDER BY n_tup_upd DESC
LIMIT 20;

-- 6. Show FILLFACTOR for all tables
SELECT
```

```

n.nspname,
c.relname,
COALESCE(
    (SELECT option_value FROM pg_options_to_table(c.reloptions)
     WHERE option_name = 'fillfactor'),
    '100'
)::int AS fillfactor
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind = 'r'
      AND n.nspname = 'public'
ORDER BY c.relname;

-- 7. Detect tables that may benefit from lower FILLFACTOR
-- Low HOT rate + high update count = candidate for lower FILLFACTOR
SELECT
    relname,
    n_tup_upd,
    n_tup_hot_upd,
    round(100.0 * n_tup_hot_upd / NULLIF(n_tup_upd, 0), 1) AS hot_pct,
    'Consider fillfactor 70-80' AS recommendation
FROM pg_stat_user_tables
WHERE n_tup_upd > 10000
      AND (n_tup_hot_upd::float / NULLIF(n_tup_upd, 0)) < 0.5
ORDER BY n_tup_upd DESC;

```

Common Mistakes

1. **Leaving FILLFACTOR at 100 for high-UPDATE tables.** Default FILLFACTOR means pages are always full → no room for HOT updates → index bloat and extra WAL on every UPDATE.
2. **Setting FILLFACTOR too low for read-heavy tables.** A FILLFACTOR of 50% means pages are half-empty, doubling the number of pages the planner needs to read for a sequential scan.
3. **Not running VACUUM after bulk deletes.** Deleted space is not reflected in the FSM until VACUUM runs. INSERT will keep extending the file instead of reusing freed pages.
4. **Changing FILLFACTOR without applying it.** `ALTER TABLE t SET (fillfactor = 70)` only affects new pages. Existing pages retain their current fill level. Use `VACUUM FULL` or `CLUSTER` to rewrite existing pages with the new FILLFACTOR (requires lock).
5. **Ignoring the `n_tup_hot_upd` statistic.** A low HOT update rate on a high-UPDATE table is a strong signal that FILLFACTOR needs tuning.
6. **Confusing FSM accuracy with exactness.** The FSM stores an approximation (free space quantised to 32-byte units). A page returned by the FSM may not actually have enough space — the backend double-checks the actual page and may try another.

Best Practices

- **Set FILLFACTOR = 70–80** for OLTP tables with frequent row-level updates.

- **Leave FILLFACTOR = 100** for append-only and analytical tables.
- **Monitor `n_tup_hot_upd / n_tup_upd`** — aim for > 80% HOT rate on high-update tables.
- **After bulk deletes or TRUNCATE**, run VACUUM to update the FSM so freed space is immediately reusable.
- **Do not disable autovacuum** — the FSM becomes stale quickly without regular VACUUM.
- **For tables with UPDATE that changes indexed columns**, HOT cannot apply; ensure indexes are truly necessary to avoid unnecessary index bloat.

Performance Considerations

Scenario	FSM Impact	Action
INSERT extending file unnecessarily	FSM shows 0 free space; pages exist with dead tuples	Run VACUUM to update FSM
Low HOT update rate	Non-indexed columns updated on full pages	Lower FILLFACTOR
Sequential scan much slower than expected	Low FILLFACTOR → many sparse pages	Consider FILLFACTOR 90–100 for read-heavy tables
Index bloat despite low UPDATE rate	HOT not triggering (indexed cols changed or pages full)	Check <code>n_tup_hot_upd</code> ; lower FILLFACTOR
FSM file very large	Many fragmented pages	Run VACUUM FULL to reclaim and rewrite

Interview Questions

Q1. What is the FSM and why does PostgreSQL need it?

The Free Space Map is a per-relation structure that records available free space per heap page. PostgreSQL uses it to quickly find a page with enough space for a new tuple without scanning all pages. Without it, INSERT would extend the file for every row even when existing pages have freed space.

Q2. How does the FSM represent free space per page?

Each heap page is represented by one byte (0–255) where the value is `floor(free_bytes / 32)`. So free space is approximated in 32-byte increments. The maximum representable value is $255 \times 32 = 8160$ bytes.

Q3. What is a HOT update and what conditions must be met for it to occur?

A HOT (Heap Only Tuple) update occurs when: (1) the new tuple version fits on the same page as the old version, and (2) none of the indexed columns were modified. The new tuple gets `HEAP_ONLY_TUPLE` flag and no new index entries are written. The index continues to point to the old slot, which redirects to the new version via the HOT chain.

Q4. What is FILLFACTOR and how does it affect HOT updates?

FILLFACTOR (0–100, default 100) is the target fill percentage for heap pages during INSERT. A FILLFACTOR of 70 means INSERT fills pages to only 70%, reserving 30% for future HOT updates. Lower FILLFACTOR → more space available for updates on the same page → more HOT updates → less index bloat.

Q5. What does VACUUM do to the FSM?

After removing dead tuples from a page, VACUUM updates the FSM leaf entry for that page to reflect the newly available free space. It then propagates the change up the FSM tree. This allows subsequent INSERTs to reuse the freed pages instead of extending the file.

Q6. Why might INSERT extend the relation file even though dead tuples exist?

Because the FSM has stale information. The FSM is only updated when VACUUM runs. If autovacuum has not run recently, the FSM still shows those pages as full (because dead tuples physically occupy space but are not counted as "free" until VACUUM reclaims them). Result: INSERT extends the file unnecessarily.

Q7. What does `n_tup_hot_upd` in `pg_stat_user_tables` measure?

The cumulative count of HOT updates on the table (updates where the new tuple version fit on the same page and no indexed column changed). A high ratio of `n_tup_hot_upd / n_tup_upd` (e.g., > 80%) means the table is benefiting from HOT updates and `FILLFACTOR` is well-tuned.

Q8. What is HOT chain pruning?

When a heap page is accessed (read for any purpose), PostgreSQL checks for dead HOT chain ancestors. If the old tuple in a HOT chain is no longer visible to any running transaction, it is immediately pruned — the dead tuple's `ItemId` is changed to `LP_REDIRECT` pointing to the live version, or `LP_UNUSED` if it is the end of a dead chain. This happens without a full VACUUM pass, keeping pages clean on access.

Exercises with Solutions

Exercise 1 — Observe FSM before and after VACUUM

```
-- Solution
CREATE EXTENSION IF NOT EXISTS pg_freespacemap;

CREATE TABLE fsm_demo (id serial, data text);
INSERT INTO fsm_demo SELECT i, repeat('x', 100) FROM generate_series(1, 10000) i;

-- Check FSM (most pages should have little free space)
SELECT round(avg(avail), 0) AS avg_free FROM pg_freespace('fsm_demo');

-- Delete half the rows
DELETE FROM fsm_demo WHERE id % 2 = 0;

-- FSM still shows 0 free (VACUUM hasn't updated it)
SELECT round(avg(avail), 0) AS avg_free_before_vacuum FROM pg_freespace('fsm_demo');

-- Vacuum
VACUUM fsm_demo;

-- FSM now shows freed space
SELECT round(avg(avail), 0) AS avg_free_after_vacuum FROM pg_freespace('fsm_demo');
-- Should be much higher now (~50% of page size)
```

Exercise 2 — Measure HOT update rate

```

-- Solution
CREATE TABLE hot_test (
    id    serial PRIMARY KEY,
    name  text,
    score int
) WITH (fillfactor = 70);

-- Create index only on id (not on name or score)
-- score and name are non-indexed → HOT updates possible

INSERT INTO hot_test (name, score)
SELECT 'player_' || i, i FROM generate_series(1, 10000) i;

-- Reset stats
SELECT pg_stat_reset();

-- Run 50000 updates on non-indexed columns
DO $$
BEGIN
    FOR i IN 1..50000 LOOP
        UPDATE hot_test SET score = floor(random() * 100)
        WHERE id = floor(random() * 10000 + 1)::int;
    END LOOP;
END;
$$;

-- Check HOT rate
SELECT
    relname,
    n_tup_upd,
    n_tup_hot_upd,
    round(100.0 * n_tup_hot_upd / NULLIF(n_tup_upd, 0), 1) AS hot_pct
FROM pg_stat_user_tables
WHERE relname = 'hot_test';
-- With fillfactor=70, hot_pct should be significantly higher than with
fillfactor=100

```

Exercise 3 — FILLFACTOR impact comparison

```

-- Solution: Compare HOT rate with different fillfactors
CREATE TABLE ff100 (id serial PRIMARY KEY, val int) WITH (fillfactor = 100);
CREATE TABLE ff70  (id serial PRIMARY KEY, val int) WITH (fillfactor = 70);

INSERT INTO ff100 SELECT i, i FROM generate_series(1, 100000) i;
INSERT INTO ff70  SELECT i, i FROM generate_series(1, 100000) i;

SELECT pg_stat_reset();

DO $$
BEGIN

```

```
FOR i IN 1..100000 LOOP
    UPDATE ff100 SET val = val + 1 WHERE id = (random() * 100000 + 1)::int;
    UPDATE ff70 SET val = val + 1 WHERE id = (random() * 100000 + 1)::int;
END LOOP;
END;
$$;

SELECT relname, n_tup_upd, n_tup_hot_upd,
       round(100.0 * n_tup_hot_upd / n_tup_upd, 1) AS hot_pct
FROM pg_stat_user_tables
WHERE relname IN ('ff100', 'ff70');
-- ff70 should show higher hot_pct
```

Cross-References

- **02_heap_storage.md** — Page header fields `pd_lower` , `pd_upper` , free space calculation
- **06_vacuum.md** — VACUUM updates FSM after removing dead tuples
- **07_autovacuum.md** — Autovacuum keeps FSM current
- **08_visibility_map.md** — VM and FSM are complementary per-page tracking structures
- **07_Indexes** — Index entries avoided by HOT updates